TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 4: Greedy Algorithms I

Foreign Trade University

Contents

1 Introduction: The Greedy Paradigm

This week begins our study of **greedy algorithms**: algorithms that build up a solution piece by piece, at each step making the choice that looks best *right now*, according to some simple local rule, and never reconsidering that choice later.

Greedy algorithms are appealing because they are usually short, fast, and easy to implement – often just "sort the input by some key, then scan once." The hard part is never the implementation; it is the **proof**. For almost any greedy rule one can imagine, there is a plausible-sounding argument for why it should work, and (often) a counter-example showing it does not. Distinguishing the two requires a careful correctness proof.

1.1 Two proof techniques

This course (following Kleinberg & Tardos, Ch. 4) uses two main templates for proving a greedy algorithm correct.

- **Greedy stays ahead.** Define a natural measure of progress (e.g., "after k steps, the greedy solution has done at least as well as any other algorithm's first k steps, according to some measure"). Prove by induction on k that the greedy solution stays "ahead of" or "tied with" every other feasible solution at every step. Conclude that, at the end, greedy is at least as good as any alternative.
- **Exchange argument.** Take an arbitrary optimal solution O that may differ from the greedy solution G. Show that O can be transformed into G by a sequence of local "exchanges" – swaps that do not make the solution any worse (and sometimes strictly improve it, or at least do not violate feasibility). Since each exchange preserves (or improves) optimality, and the sequence of exchanges turns O into G, G must also be optimal.

Both techniques appear repeatedly in this course. Today we will see the exchange argument in its cleanest forms, applied to interval scheduling and to minimizing lateness.

1.2 Why naive greedy rules often fail

A recurring theme today is that *many plausible greedy rules are wrong.* For a problem with several "obvious" greedy orderings (by size, by deadline, by start time, by value, ...), usually only one (or a specific combination) is provably correct, and the others fail on small counter-examples. Part of learning to design greedy algorithms is learning to quickly construct such counter-examples – which is exactly what several of this week's exercises ask you to do.

2 Interval Scheduling

2.1 Problem statement

Definition 2.1 (Interval Scheduling Problem)**.** *We are given a set of* n requests *(or* jobs*)* $1, 2, \ldots, n$*, where request* i *has a* start time s_i *and a* finish time f_i*, with* $s_i < f_i$*. Two requests* i *and* j *are* compatible *if the intervals* $[s_i, f_i)$ *and* $[s_j, f_j)$ *do not overlap (i.e.,* $s_i \geq f_j$ *or* $s_j \geq f_i$*). The goal is to select a* ***maximum-size*** *subset of mutually compatible requests.*

Think of each request as a meeting that needs a single shared room: we want to schedule as many meetings as possible in that one room.

2.2 Why several natural greedy rules fail

Before presenting the correct algorithm, it is worth listing some tempting rules and why each one can fail:

- **Earliest start time first.** Counter-example: one request starting very early but lasting all day blocks out many short requests that could otherwise all fit.
- **Shortest interval first.** A short interval can overlap (and thus eliminate) two or more longer, mutually compatible intervals – Problem 6 in this week's exercises asks you to construct exactly such an instance.
- **Fewest conflicts first.** Plausible, but the bookkeeping is complex and (perhaps surprisingly) it is not needed: a much simpler rule turns out to be optimal.

2.3 The earliest-finish-time-first algorithm

Algorithm 1 Earliest-Finish-Time-First (Interval Scheduling)

```
1: Sort requests by finish time: f_1 ≤ f_2 ≤ ··· ≤ f_n
2: A ← ∅
3: for i = 1 to n do
4:     if request i is compatible with every request currently in A then
5:         A ← A ∪ {i}
6:     end if
7: end for
8: return A
```

Because requests are processed in order of finish time, "compatible with everything in A" simplifies to "starts no earlier than the finish time of the most recently added request" – so the inner check is $O(1)$ given the previous selection, and the whole algorithm runs in $O(n \log n)$ (dominated by the sort). Problem 3 asks you to implement exactly this.

2.4 Correctness: an exchange argument

Theorem 2.2. *The earliest-finish-time-first algorithm produces a maximum-size set of mutually compatible requests.*

Proof. Let $i_1, i_2, \ldots, i_k$ be the requests selected by the greedy algorithm, in the order selected (so $f_{i_1} < f_{i_2} < \cdots < f_{i_k}$, and f_{i_1} is the smallest finish time among *all* requests). Let $j_1, j_2, \ldots, j_m$ be the requests of an arbitrary optimal solution, also sorted by finish time. We show $k \geq m$, which (since the greedy solution is always feasible) implies $k = m$.

Claim (greedy stays ahead). For every r with $1 \leq r \leq \min(k, m)$, $f_{i_r} \leq f_{j_r}$.
Proof of claim, by induction on r.

- **Base case ($r = 1$).** i_1 was chosen because it has the globally smallest finish time among *all* n requests. In particular $f_{i_1} \leq f_{j_1}$.
- **Inductive step.** Suppose $f_{i_{r-1}} \leq f_{j_{r-1}}$. Since j_r is compatible with j_{r-1} in the optimal solution, $s_{j_r} \geq f_{j_{r-1}} \geq f_{i_{r-1}}$. So j_r is compatible with i_{r-1} – and hence also compatible with every $i_1, \ldots, i_{r-1}$ (which all finish no later than $f_{i_{r-1}}$). Therefore, when the greedy algorithm

considers requests in increasing order of finish time, j_r (or some other request with finish time $\leq f_{j_r}$) was a *feasible* candidate to extend $i_1, \ldots, i_{r-1}$ at the point where the greedy algorithm chose i_r. Since greedy picks the smallest-finish-time compatible request available, $f_{i_r} \leq f_{j_r}$.

Finishing the proof. Suppose for contradiction $k < m$. Apply the claim with $r = k$: $f_{i_k} \leq f_{j_k}$. Since j_{k+1} exists (because $m > k$) and is compatible with j_k, we have $s_{j_{k+1}} \geq f_{j_k} \geq f_{i_k}$, so j_{k+1} is compatible with $i_1, \ldots, i_k$ – meaning the greedy algorithm *could* have added j_{k+1} (or some request with an even smaller finish time) after i_k, contradicting the fact that greedy's selection ended at i_k (greedy only stops adding a request when no compatible one remains, but j_{k+1} was compatible). Hence $k \geq m$, and since A is feasible, $k \leq m$ as well (an optimal solution has size $\geq$ any feasible solution), so $k = m$. □

Problems 4–5 ask you to verify this empirically: implement a brute-force optimum for small instances and confirm the greedy result always has the same size.

3 Interval Partitioning

3.1 Problem statement

Definition 3.1 (Interval Partitioning Problem)**.** *Given n requests (intervals) as before, partition them into the* minimum *number of* resources *(e.g. classrooms) such that no two requests assigned to the same resource overlap. Equivalently: find the minimum number of "colors" needed to color the intervals so that overlapping intervals get different colors.*

This differs from Interval Scheduling: here *every* request must be scheduled (on some resource), but we get to use as many resources as needed – we just want to minimize that number.

3.2 A lower bound: depth

Definition 3.2 (Depth)**.** *The* depth *of a set of intervals is the maximum, over all points in time t, of the number of intervals containing t.*

Lemma 3.3. *No assignment of intervals to resources (with no overlaps on a single resource) can use fewer than* depth *resources.*

Proof. If $d =$ depth, then there is some time t contained in d distinct intervals. These d intervals are pairwise overlapping (they all contain t), so no two of them can share a resource. Hence at least d distinct resources are required. □

3.3 The earliest-start-time-first algorithm

Using a binary min-heap keyed by end time, each interval requires $O(\log n)$ heap operations, so the whole algorithm runs in $O(n \log n)$. Problem 7 asks you to implement this directly.

3.4 Correctness: the greedy uses exactly depth resources

Theorem 3.4. *The earliest-start-time-first algorithm uses exactly $d =$ depth resources, and this is optimal.*

Algorithm 2 Earliest-Start-Time-First (Interval Partitioning)

Sort intervals by start time: $s_1 \le s_2 \le \cdots \le s_n$
$d \leftarrow 0$ ▷ number of resources allocated so far
$H \leftarrow \emptyset$ ▷ min-heap of (end_time, resource_id) for "busy" resources
for $i = 1$ to n **do**
    if $H \neq \emptyset$ and $\min(H)$.end_time $\le s_i$ **then**
        Pop the resource r with smallest end_time from H
        Assign interval i to resource r
    else
        $d \leftarrow d + 1$; assign interval i to a new resource d
    end if
    Push $(f_i, \text{resource of } i)$ onto H
end for
return d and the assignment

Proof. By Lemma 3.3, no algorithm can use fewer than depth resources, so it suffices to show the greedy algorithm uses *at most* depth resources.

Suppose the greedy algorithm allocates a new resource (resource number d) when processing interval i. This happens only when, at the moment interval i is considered, *every* resource $1, \ldots, d-1$ currently holds an interval that has not yet finished, i.e. for each resource $r < d$ there is some interval j_r with $s_{j_r} \le s_i < f_{j_r}$ (it started no later than s_i and finishes after s_i). Together with interval i itself (which also "contains" s_i, since $s_i < f_i$), this gives d intervals – the $d-1$ intervals $j_1, \ldots, j_{d-1}$ plus interval i – all containing the point s_i. Hence depth $\ge d$.

So whenever the greedy algorithm uses a resource numbered d, we have $d \le$ depth. The total number of resources used is therefore at most depth, and combined with the lower bound, it equals depth exactly. □

Problem 8 asks you to verify this equality empirically on random instances, and Problems 9–10 explore related feasibility questions (can k resources suffice?) and stress-test the implementation.

4 Scheduling to Minimize Lateness

4.1 Problem statement

Definition 4.1 (Minimizing Lateness Problem)**.** *We are given n jobs, all available for processing at time 0. Job i has a* processing time *(length) $t_i > 0$ and a* deadline *d_i. A single resource processes jobs one at a time, without preemption (once started, a job runs to completion). For a schedule, let f_i be the finish time of job i. The* lateness *of job i is $\ell_i = \max(0, f_i - d_i)$. The goal is to find a schedule minimizing $L = \max_i \ell_i$, the* ***maximum lateness****.*

4.2 Why naive rules fail here too

- **Shortest job first (ignoring deadlines).** A very short job with a very tight deadline can be starved by a slightly longer job with no urgency, if the longer job is scheduled first merely because... no wait, shortest-first would schedule the short job first in that case. The real failure mode: a job with a generous deadline but very short length gets scheduled first, "wasting" the early time slots that a job with a tight deadline desperately needed. Problem 14 asks you to construct such an instance.

- **Smallest slack first (i.e., $d_i - t_i$ smallest first).** This rule is actually used for some related problems but can be shown to fail here too in certain edge cases – we will not pursue it further in this course.

4.3 The earliest-deadline-first algorithm

Algorithm 3 Earliest-Deadline-First (Minimizing Lateness)

```
Sort jobs by deadline: d_1 ≤ d_2 ≤ ··· ≤ d_n
t ← 0
for i = 1 to n do
    Schedule job i to run in [t, t + t_i)
    f_i ← t + t_i
    t ← f_i
end for
return the schedule and L = max_i max(0, f_i − d_i)
```

This is exactly "sort by deadline, then run them back to back" – $O(n \log n)$ total. Problem 11 asks you to implement this.

4.4 Two structural facts about optimal schedules

Definition 4.2 (Idle time and inversions)**.** *A schedule has* idle time *if the resource is idle at some time t while some job has not yet been completed (and could have started by time t). A schedule (given as an ordering of jobs) has an* inversion *if there exist jobs i, j with i scheduled before j but $d_i > d_j$.*

Lemma 4.3. *There exists an optimal schedule with no idle time and, among schedules with no idle time, an optimal one exists with no inversions.*

Proof sketch. **No idle time.** If a schedule has idle time, we can shift every job after the idle period earlier (removing the gap) without increasing any finish time, hence without increasing any lateness. So removing idle time never makes the schedule worse.

No inversions (exchange argument). Suppose an idle-free schedule has an inversion: some adjacent pair of jobs i (scheduled right before j) with $d_i > d_j$ (any inversion must contain an adjacent inverted pair, by a standard argument – if i before j is an inversion but they are not adjacent, there is some adjacent inverted pair between them). Swap i and j. All other jobs' finish times are unchanged. Job j now finishes earlier than before (it moved earlier in the order), so its lateness can only decrease. Job i now finishes at the time j used to finish, call it f. We claim job i's new lateness $\max(0, f - d_i)$ is *at most* the larger of the two jobs' old latenesses. Concretely, before the swap, job j (the later one) finished at time f, so its old lateness was $\max(0, f - d_j)$. Since $d_i > d_j$, we have $f - d_i < f - d_j$, so $\max(0, f - d_i) \le \max(0, f - d_j)$ – job i's new lateness is no worse than job j's old lateness. Hence the maximum lateness over the whole schedule does not increase after the swap. Repeating this for every inversion (each swap strictly reduces the inversion count) eventually yields an inversion-free schedule that is no worse than the original. □

Theorem 4.4. *The earliest-deadline-first algorithm produces a schedule with the minimum possible maximum lateness.*

Proof. By Lemma 4.3, some optimal schedule O has no idle time and no inversions. A schedule with no idle time and no inversions must process jobs in non-decreasing order of deadline (any other order would contain an inversion) – which is *exactly* the order produced by earliest-deadline-first. Since all idle-free schedules with the same job order have identical finish times (jobs run back-to-back starting at time 0), O and the earliest-deadline-first schedule are the same schedule (up to tie-breaking among jobs with equal deadlines, which does not affect L). Hence earliest-deadline-first achieves the optimal L. □

Problems 12–13 ask you to verify this against a brute-force search over all $n!$ orderings for small n, and Problem 15 asks you to directly measure inversions in EDF's output (it should always be zero) and in perturbed orderings.

5 Optimal Caching

5.1 Problem statement

Definition 5.1 (Optimal Caching Problem)**.** *We are given a cache that can hold up to k items, and a sequence of m requests $\sigma = \sigma_1, \sigma_2, \ldots, \sigma_m$, each requesting some item. If a requested item is already in the cache, this is a* hit *(free); otherwise it is a* miss*, and if the cache is full, some item currently in the cache must be* evicted *to make room. The goal (in the* offline *version, where the entire sequence σ is known in advance) is to choose an eviction policy minimizing the total number of misses.*

This models, e.g., a CPU cache, a web browser's page cache, or a database buffer pool: k is the cache capacity, and each "miss" represents an expensive operation (a memory fetch, a disk read, a network request) that a hit avoids.

5.2 The furthest-in-future algorithm

Algorithm 4 Furthest-In-Future (Belady's MIN / LFD)

```
for each request σ_i, in order do
    if σ_i is already in the cache then
        (hit; do nothing)
    else
        if the cache is full then
            Evict the item in the cache whose next occurrence in σ_{i+1}, σ_{i+2}, ... is furthest in the future (or that never occurs again)
        end if
        Load σ_i into the cache (miss)
    end if
end for
```

Theorem 5.2 (Belady, 1966)**.** *Furthest-in-future is an optimal offline caching policy: it achieves the minimum possible number of misses on any request sequence.*

Proof sketch – exchange argument. Let FF denote the furthest-in-future schedule of evictions, and let O be any optimal eviction schedule. We show O can be transformed, step by step, into a schedule that behaves identically to FF, without increasing the number of misses, via the following

idea: consider the first point at which FF and O make a different eviction decision on a miss. At that point, FF evicts the item x used furthest in the future; suppose O evicts some other item y (so y's next use is sooner than x's). We can modify O to evict x instead of y at this step, and show that O's subsequent behavior can be adjusted (by "pretending" to still have y in cache until it is needed, then evicting whatever FF would have evicted at that later point) so that the total number of misses does not increase. This is the technique called "reduction to a canonical form" – repeating this argument shows some optimal schedule agrees with FF at every step, hence FF itself is optimal. The full details are in Kleinberg & Tardos, Section 4.4. □

5.3 Online caching: LRU

Furthest-in-future requires knowing the *future* request sequence – it is an *offline* algorithm. In practice, caches must make decisions *online*, without knowing future requests. The most common online heuristic is **Least-Recently-Used (LRU)**: evict the item whose most recent access is furthest in the *past*, on the heuristic assumption that recently-used items are likely to be used again soon ("temporal locality").

LRU can be far from optimal: Problem 18 asks you to construct a request sequence where LRU incurs strictly more misses than furthest-in-future, by exploiting a cyclic access pattern that defeats the "recently used = likely reused" heuristic. (In the worst case, LRU can be forced to miss on *every single request* in a cyclic pattern over $k+1$ items with a cache of size k, while furthest-in-future does strictly better.)

6 A Different Flavor: Greedy on Coin Change and Job Sequencing

Not every greedy algorithm in this chapter is about intervals. Two more classical examples, explored in Problems 20–22:

6.1 Coin change

Given coin denominations (e.g. $\{1, 2, 5, 10, 20, 50, 100, 200, 500\}$) and a target amount, the **greedy** algorithm repeatedly takes the largest denomination that does not exceed the remaining amount. For *canonical* coin systems (which include all standard currency denominations in common use), this greedy algorithm uses the minimum possible number of coins. However, this is *not* true for arbitrary denomination sets: with denominations $\{1, 3, 4\}$, greedy makes change for 6 as $4 + 1 + 1$ (3 coins), while $3 + 3$ (2 coins) is strictly better. Whether a coin system is canonical is, in general, a non-trivial property – proving it requires checking a finite (but possibly large) set of "witness" amounts.

6.2 Job sequencing with deadlines and profits

Each job i has a deadline d_i and a profit p_i, takes one unit of time, and earns p_i only if scheduled at or before d_i on a single machine (with time slots $1, 2, 3, \ldots$). The greedy algorithm processes jobs in *decreasing order of profit*, and places each job in the *latest* available slot at or before its deadline (if any slot is available). The intuition: placing a job as late as possible "wastes" the fewest early slots, leaving them open for other jobs with tighter deadlines. This greedy algorithm is optimal, and can be proven correct with another exchange argument (swapping a job into a later free slot never decreases total profit, since both jobs still meet their deadlines).

7 Summary and Looking Ahead

This week introduced the **greedy algorithm design paradigm** and two proof techniques – *greedy stays ahead* and the *exchange argument* – through four case studies:

- **Interval Scheduling**: earliest-finish-time-first is optimal (greedy stays ahead).
- **Interval Partitioning**: earliest-start-time-first uses exactly depth resources, which is optimal.
- **Minimizing Lateness**: earliest-deadline-first is optimal (exchange argument via inversions).
- **Optimal Caching**: furthest-in-future is optimal offline (exchange argument / reduction to canonical form); LRU is a practical but suboptimal online heuristic.

We also saw, repeatedly, that *not every natural-sounding greedy rule is correct* – shortest-interval-first and shortest-job-first both fail, and the non-canonical coin system $\{1, 3, 4\}$ shows that even "obviously correct" greedy rules (like coin change) can fail outside their intended domain. Constructing counter-examples is as important a skill as designing the correct algorithm.

Next week (Greedy II, Kleinberg & Tardos Ch. 4 continued) we apply the same exchange-argument toolkit to two fundamental graph algorithms: Dijkstra's shortest-path algorithm and minimum spanning trees (Kruskal's and Prim's algorithms).